Can Language Models Tell When a Robot Is Done? A Benchmark for Supervising Learned Robot Policies

Tom Wang Ranveer Singh Ty Lipscomb

COS 597C, Project 1 report, October 26, 2026

Code, data, results and exact commands: `https://github.com/tom05919/OrcaBench`

Abstract

Robot policies learned from demonstrations do not stop when they have finished a task; they keep moving until a time limit. One fix is to let a vision-language model watch the robot's cameras and say when the task is done. This supervisor faces a trade-off: every look at the cameras costs time, but stopping too early fails the task. We built a benchmark for this skill. It replays 168 recorded simulation videos of three learned policies doing kitchen tasks; at each decision the model sees the camera images and either waits a number of steps or declares the task done, and the simulator's built-in success check scores the decision automatically. An episode is worth more the sooner the task is correctly declared done and the fewer looks it took, and nothing if "done" comes too early. The headline *supervision score* is the share of the best achievable worth that a model keeps, where 100 is an oracle that stops at the exact moment the robot finishes. We evaluated four open vision-language models, each with reasoning off, low and high. The best configuration scores 40 (95% CI 24–53). In paired comparisons on the same runs, every configuration scores below a rule that sees perfectly but checks at a fixed interval (61), and 11 of 12 score below a timer that never looks at the cameras (51). The main cause is stopping too early: models declare the task done before the robot has finished on 18–62% of the runs it does finish. A separate test of single "is it finished?" questions shows the same weakness (69–78% balanced accuracy). Turning reasoning on helps two of the four models answer these questions but does not lift any of them to the simple baselines.

1 Motivation

Vision-language-action policies such as GR00T [2] and $\pi_{0.5}$ [3] control a robot arm from camera images and a text instruction. They are trained to imitate demonstrations, and they have no notion of having finished: in every recording we use, the policy keeps acting until the episode's time limit, including after it has completed the task. Something has to decide when to stop. A natural candidate is a vision-language model acting as a *supervisor*: it looks at the robot's cameras from time to time and declares the task done, as language models already plan and monitor robot skills in prior work [4, 5].

A good supervisor needs two skills. It must **recognise completion** from camera images, which is hard because a task that is almost done looks very much like a task that is done. It must also **decide when to look**: each look costs time (the model has to be called and answer) and money, so looking constantly is wasteful, while looking rarely means stopping long after the robot has finished, and stopping before it has finished fails the task. Vision-language models have been tested as success detectors on curated video clips [6], but we know of no benchmark that scores the whole supervision decision, *when to look and when to stop*, against simple non-language baselines.

Contributions. (1) A benchmark that scores both skills automatically on recorded runs of real learned policies, with a single headline score and baselines that use no language model. (2) Results for four open vision-language models with reasoning off, low and high (12 configurations), with 95% confidence intervals. (3) An account of what did not work, including the first version of this benchmark.

2 The benchmark

Recorded robot runs. We recorded three policies from the official RoboCasa365 [1] release (GR00T N1.5 trained on many tasks, GR00T N1.5 fine-tuned on 30% of the target tasks' demonstrations, and $\pi_{0.5}$) on seven kitchen tasks in four simulated kitchens, twice each: 168 runs. Three tasks are ones the policies were trained on (open a cabinet, move an object from the counter to the stove, close the fridge); the robot finishes 50 of these 72 runs. The other four are new multi-step tasks that it finishes in only 7 of 96 runs. Each run has three 256×256 camera videos and the simulator's success check at every control step (20 steps per second). The success check is used only for scoring; the model never sees it.

Why replay works. The supervisor can only say "wait" or "done". Neither changes how the robot moves, so a recording of the robot running uninterrupted shows exactly what the supervisor would have seen live. Every model therefore sees identical runs, experiments are repeatable and cheap, and no GPU is needed.

Supervision test (the main test). One *episode* supervises one recorded run. At the start, the model sees the task instruction and the camera images and replies either `wait` N (10 to 400 steps) or `done`. After a wait it sees new images (now, and two earlier moments of the interval it skipped) and decides again. Each such decision is a *camera check*. A "done" succeeds if the success check is true at that step; otherwise the episode fails, as it does if the time limit runs out. Every check is charged 2 s, the same for every model, so that the provider's server speed does not affect the score. An episode's value is

$$\text{value} = \text{success} \times \max\Big(0,\ 1 - \frac{\text{robot time} + 2\,\text{s} \times \text{checks}}{\text{time limit}}\Big),$$

where robot time is how long the robot has run when the model says "done" and the time limit is the task's budget. Stopping soon after the robot finishes, with few checks, scores high;

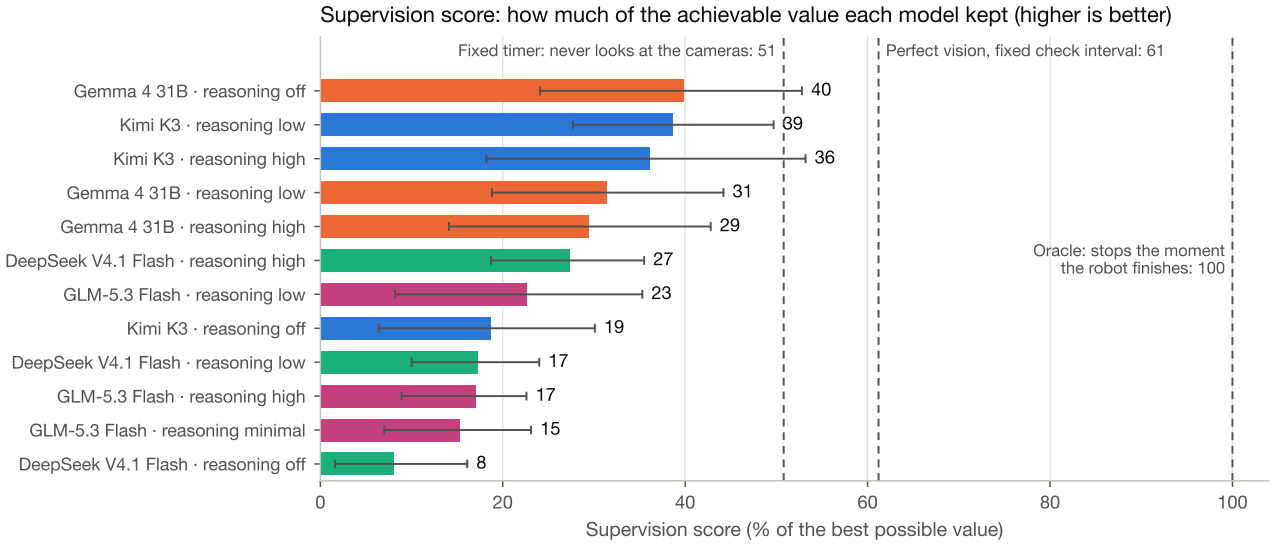


Figure 1: Supervision leaderboard. Each bar is one configuration's supervision score: its value on the 72 trained-task episodes as a percentage of an oracle that stops at the exact moment the robot finishes (100). Dashed lines: a timer that never looks at the cameras (51) and a rule with perfect vision that checks at a fixed interval (61), both tuned on other kitchens. Whiskers: 95% bootstrap intervals. GLM-5.3 Flash's lowest setting is "minimal".

failing scores 0. Episodes run in blocks of 21 per kitchen (seven tasks $\times$ three policies, each policy named only by a letter); before each episode the model sees one line per earlier episode in the block (policy letter, task, outcome), so it can learn how fast each policy usually is. *Example* (Gemma 4, reasoning low, close the fridge): the model waited 100, 200 and 200 steps, then said "done" at step 500; the robot had finished at step 493, so the episode succeeded with 4 checks and value $1 - (25 + 4 \times 2)/45 = 0.27$.

Supervision score. We report the model's total value as a percentage of the oracle's total value on the 72 trained-task episodes:

- **Oracle** (100): stops at the exact step the robot finishes. Its absolute value is only 0.28 on average, because the robot itself uses much of the time limit and 22 of the 72 runs never finish; dividing by it makes 100 the best any supervisor could do on these recordings.
- **Perfect vision, fixed check interval** (61): a rule that always knows whether the task is finished but checks every k steps, with k tuned on the same policy and task in the other kitchens. It loses value only to the cost of its checks and to noticing completion up to k steps late. It separates "cannot see that it is done" from "checks at the wrong times".
- **Fixed timer** (51): never looks at the cameras; declares "done" after a fixed number of steps, tuned the same way. It represents supervision with no perception at all.

The new tasks are excluded from the score because the robot almost never finishes them, so every method, the oracle included, scores close to 0 there (all results are in the repository).

Done-detection test. To measure perception alone, we ask 450 single questions with no history: the task instruction and the camera images at one moment (plus 2 s and 5 s earlier), "is the task finished?". 114 moments are finished (the moment the robot finishes and 5 s later) and 336 are not (10 s and 2 s before it finishes, and moments of runs that never finish). Moments in the last 2 s before finishing are not used, because the success check lags the images by about 1 s. The *done-detection score* is balanced accuracy, the mean of the accuracy on finished and on unfinished moments (50 = chance), because plain accuracy would reward always answering "not finished".

Decision	Reason
Replay recorded runs	Waiting and stopping do not change the robot; identical inputs for every model, repeatable, no GPU.
Separate done-detection test	Measures perception without scheduling, to explain supervision results.
Labels from the simulator's success check	Fully automatic scoring; no human labels.
Same 2 s charge per check for every model	Measured latency on a shared API reflects server load, not the model.
Baselines tuned in other kitchens	A rule must not be tuned on the run it is scored on.
Score as % of the oracle	One number a reader can interpret: the share of the achievable value kept.
Earlier outcomes shown, policies anonymous	Lets a model learn each policy's speed without revealing the current run's outcome.
Same reasoning settings and limits for every model; reasoning length recorded	A uniform design; the recorded length checks that a setting actually changed the reasoning.
95% bootstrap intervals resampling runs	Answers about the same run are correlated.

Table 1: Design decisions and their reasons.

Table 1 lists each design decision with its reason; all were fixed before the runs they govern.

3 Experimental setup

Models. Kimi K3, Gemma 4 31B, DeepSeek V4.1 Flash and GLM-5.3 Flash, the four open models that accept images through the Sail Research API, which serves quantized weights; results are for those deployments. Every model received the same prompts, a 32,768-token reply limit (no reply reached it) and the same retry rules.

Reasoning settings. Each model ran with reasoning *off*, *low* and *high*, the provider's reasoning-effort setting. GLM-5.3

Flash cannot turn reasoning off; its lowest setting, *minimal*, replaces off. A model with a setting is a *configuration*; each of the 12 configurations answered all 450 done-detection questions and supervised all 168 runs once. Intervals are 95% bootstrap intervals resampling recorded runs (done-detection) or task–kitchen pairs (supervision). The 12 configurations cost an estimated $72 in API fees.

4 Results

4.1 No model supervises as well as simple rules

Figure 1 ranks the 12 configurations. The best, Gemma 4 31B with reasoning off (40) and Kimi K3 with reasoning low (39), keep about two fifths of the achievable value; the worst, DeepSeek V4.1 Flash with reasoning off, keeps 8. Comparing each configuration with the baselines on the same runs, all 12 are significantly below the perfect-vision rule, and 11 are significantly below the fixed timer; Kimi K3 with reasoning high is the one we cannot distinguish from the timer. On average, a supervisor that never looks at the robot did better than every one of these configurations.

4.2 Why: models stop too early

Figure 2 shows where the value goes. On runs the robot does finish, every configuration says "done" too early on 18–62% of them, and each early "done" scores 0; the perfect-vision rule never does. The models check about as often as that rule (4–9 times against its 6) and, when they are right, are about as late (median 2–6 s against its 4 s), except two configurations with reasoning off: Kimi K3 and DeepSeek V4.1 Flash check 9 times and are 10 and 18 s late. Turning reasoning on makes these two models check less and stop sooner, which raises their score (Kimi K3 from 19 to 39) but also makes them stop too early more often (24% to 42% for Kimi K3). On the 22 runs the robot never finishes, every configuration eventually says "done"; those episodes score 0 either way. We also tested whether models adapt their checks to the policies they learn about within a block (a slower policy should get a longer first wait); we found no consistent evidence of adaptation, though this test has little statistical power.

4.3 Recognising completion is the bottleneck

The done-detection test (Figure 3) isolates perception. Every configuration is between 69 and 78 balanced accuracy, and the intervals overlap heavily, so the test does not separate the models much. Their errors are where the supervision failures are: shown the cameras 2 s before the robot finishes, configurations answer "finished" 62–80% of the time, and on moments from runs that never finish they answer "finished" 36–50% of the time. Turning reasoning on raises the score of Kimi K3 by 6.4 points (95% CI 1.3 to 11.4, same questions) and of DeepSeek V4.1 Flash by 5.8 (0.6 to 10.9), mostly by saying "finished" more readily when it is finished; it does not detectably change Gemma 4 (-0.6, -3.9 to 2.6) or GLM-5.3 Flash ($+3.2$, -0.7 to 7.2).

4.4 A reasoning setting is not a reliable treatment

We recorded how much each configuration reasoned (median characters of reasoning per done-detection answer). Only Kimi K3 has three distinct levels (0, 343 and 1,576 characters). Gemma 4 (0, 838, 844) and DeepSeek V4.1 Flash (0, 1,556, 1,630) reason the same amount at low and at high, and GLM-5.3 Flash's low setting produces no reasoning at all, exactly like minimal (0, 0, 206). Benchmarks that vary "reasoning effort" through a provider's setting should measure the reasoning they actually obtain rather than trust the label.

5 What did not work

Our first benchmark could not be interpreted. Before this design, we ran language-model supervisors (Claude Opus 5.5, Claude Sonnet 5.5 and later Kimi K3) live on GPU machines, supervising the same three policies. The supervisor could also rewrite the policy's instruction or restart it, each model call was charged that model's measured response time, and the score compared each model with a fixed stopping rule. Both Claude models lost to the rule (by 0.049 and 0.066 on a 0–1 value scale, both 95% intervals above zero) and could not be told apart, but the number did not say why. Four design problems caused this, and each shaped the benchmark above:

- **One score mixed three skills** (rewriting instructions, recognising completion, deciding when to look), so a loss could come from any of them. We now measure completion and timing, and test perception separately.
- **Rewriting instructions could not help.** Even a rule that reads the simulator's state and gives the policy correct step-by-step instructions solved 3 of 76 episodes on the new tasks, against 4 of 76 for the unchanged policy. We dropped instruction rewriting, which is what makes replay possible.
- **Charging each model's measured response time decided the result**: with no charge per call (a post-hoc check), Opus and Kimi K3 tied the rule. Response time on a shared API measures the provider's servers, so every model now pays the same 2 s per check.
- **Most episodes carried no signal.** On the new tasks the robot almost never succeeds, so 12 of 21 episodes per block scored about 0 for every method. Live runs also gave every model a different robot run, and only one pass per model was affordable. Replay removes both problems; the headline score uses the trained tasks.

Reasoning was not controlled at first. Our first replay runs sent no reasoning setting, so each model used the provider's undocumented default, which turned out to differ by model (Gemma 4 does not reason by default; the others do): model differences were confounded with reasoning. We discarded those runs from the comparison and re-ran every model with explicit settings, recording the reasoning length. We also first chose each model's settings with a 10-question pilot, which gave models different sets of settings (and was wrong for DeepSeek); we replaced it with the same three settings for every model. Operational problems (provider errors, crashes, cost accounting) and every lost or repeated API call are documented in the repository; every configuration was completed.

6 Limitations and conclusion

Each configuration ran once, so intervals reflect variation across robot runs, not across repeated model runs. The models are quantized deployments of one provider. The score rests on 72 episodes per configuration (50 runs the robot finishes) from

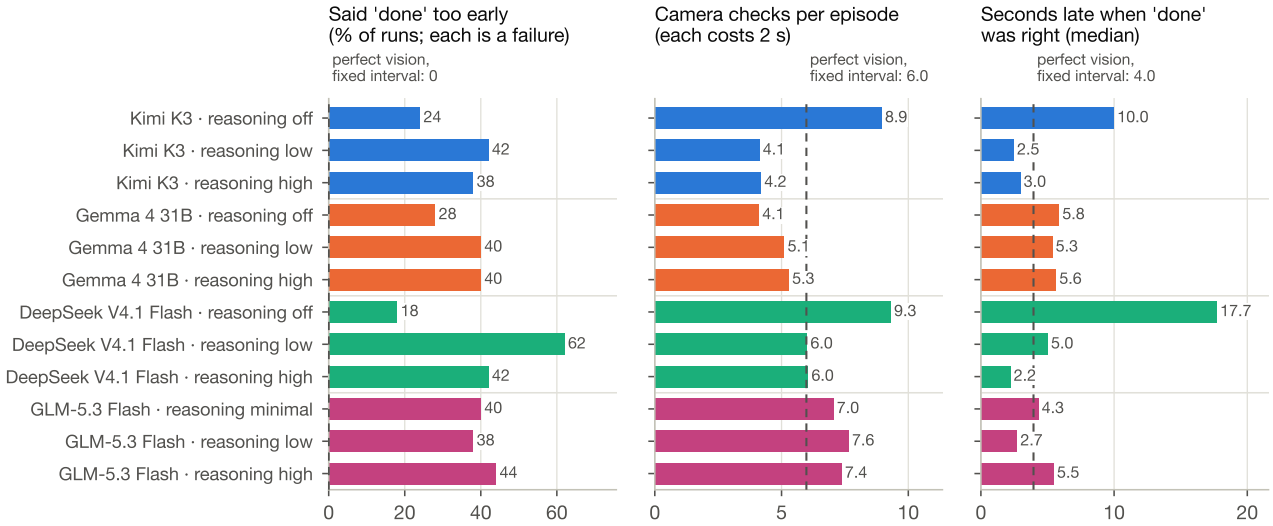


Figure 2: What the models did, on the 50 trained-task runs the robot finishes. Left: share of runs where the model said "done" before the robot had finished (each one is a failure). Middle: camera checks per episode (2 s each). Right: median seconds between the robot finishing and a correct "done". Dashed lines: the rule with perfect vision and a fixed check interval.

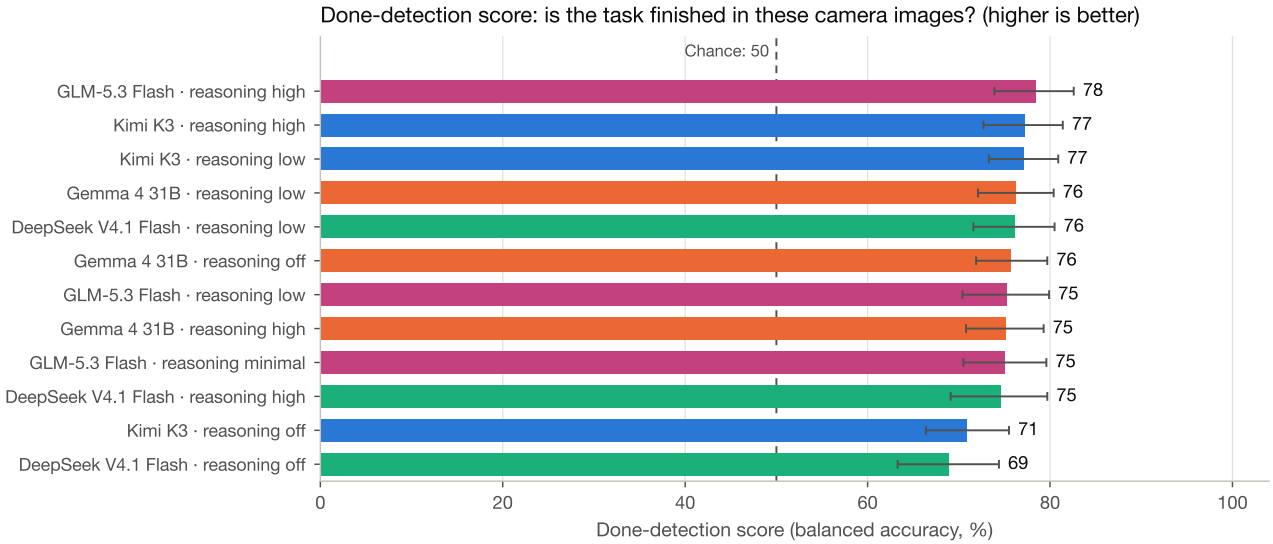


Figure 3: Done-detection leaderboard. Balanced accuracy on 450 single "is the task finished?" questions (114 finished moments, 336 unfinished); 50 is chance. Whiskers: 95% bootstrap intervals.

three trained tasks, and supervisors see images only, every 10 steps at most. A "done" given in the second before the success check turns true may be a correct reading of the image. Despite these limits, the result is clear: current open vision-language models stop learned robot policies too early, and a simple timer or a perfect-vision rule with a fixed interval does better. Reproducing every number needs no API calls: the recorded runs and all model replies are in the repository's release `replay-data-2026-10-26`.

References

[1] S. Nasiriany et al. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. RSS, 2024.

[2] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. arXiv:2503.14734, 2025.

[3] Physical Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization. arXiv:2504.16054, 2025.

[4] M. Ahn et al. Do as I can, not as I say: Grounding language in robotic affordances. CoRL, 2022.

[5] W. Huang et al. Inner monologue: Embodied reasoning through planning with language models. CoRL, 2022.

[6] Y. Du et al. Vision-language models as success detectors. CoLLAs, 2023.